



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA



DEPARTAMENTO DE
ELECTRONICA

Tarea 2

Simulador Gráfico de ELOTelTags y Aplicación Find My

Fecha 18 de mayo de 2026

Integrantes: Rodrigo Hernández
Ignacio Torres

Asignatura: Diseño y Programación Orientados a Objetos

Profesor: Agustin Gonzalez

Índice

1	Documentación	3
1.1	Interacción entre clases durante la ejecución	3
1.2	Diagrama de Clases	4
1.3	Dificultades y Soluciones	5

Índice de figuras

1. Documentación

1.1. Interacción entre clases durante la ejecución

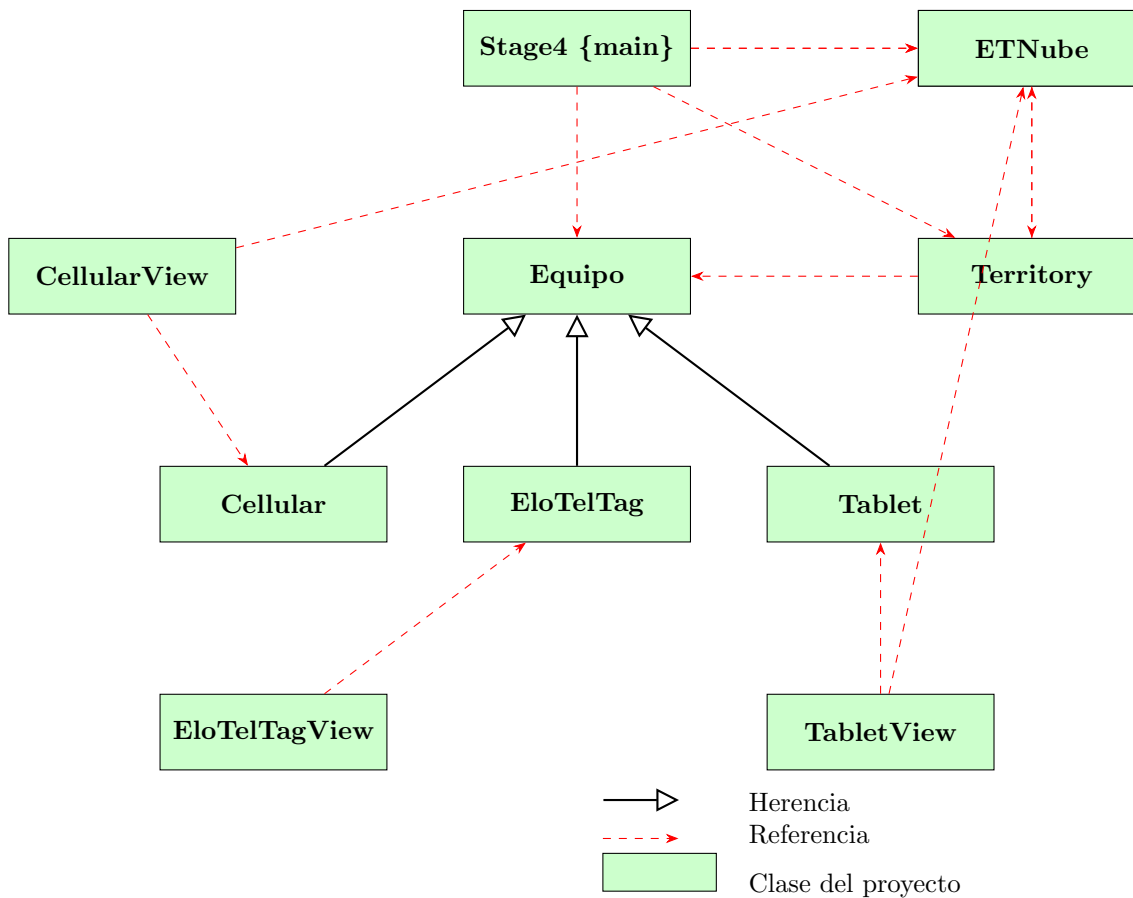
Cuando se ejecuta el programa, **Stage4** es lo primero que arranca. Le pide al usuario que elija el archivo de configuración usando un **FileChooser**, y con un **Scanner** va leyendo la imagen de fondo, el paso de tiempo y los datos de cada persona. Con esa info crea los objetos **Territory**, **TerritoryView**, y por cada persona arma su **Cellular**, sus **EloTelTag** y su **Tablet** si es que tiene, junto con las vistas de cada uno.

Una vez que el usuario le da Play en el menú Simulation, se activa un **Timeline** que cada **deltaTime** segundos llama a **updateSystem()** en **Territory**. Ahí es donde pasa todo: primero mueve todos los equipos con **move()**, que actualiza las coordenadas y hace que reboten si llegan al borde. Como las vistas están conectadas al modelo con bindings de JavaFX, se mueven solas sin necesidad de actualizarlas manualmente. Después, **Territory** revisa sus contadores de tiempo. Cada 4 segundos los celulares reportan su posición a **ETNube** con **reportSelfLocation()**. Al mismo tiempo, recorre todos los equipos buscando tags o tablets que estén a menos de 50 píxeles de algún celular. Si un **EloTelTag** está en rango y toca su ciclo de 4 segundos, el celular cercano reporta la posición del tag a la nube y se activa su propiedad **bipping**. Con las tablets pasa lo mismo pero cada 5 segundos. Cuando **bipping** cambia a true, un listener en **EloTelTagView** o **TabletView** lo detecta y lanza **mostrarRadar()**: un círculo animado que crece de 0 a 50 píxeles en 1 segundo, y si hay un archivo **bip.wav** en la carpeta, también suena.

Si se hace click en un celular o tablet, aparece un menú con la opción FindMy. Al seleccionarla se abre una ventana nueva que consulta **ETNube** y muestra dónde están los bienes de esa persona. Un **Timeline** interno refresca esa ventana cada 1 segundo para que las posiciones se vayan actualizando. Cuando se cierra la ventana, el **Timeline** se detiene.

1.2. Diagrama de Clases

A continuación se presenta el diagrama de clases de la aplicación en su etapa final (Etapa 4), que muestra las relaciones de herencia y referencia entre las clases del proyecto.



El diagrama muestra cómo está organizado el simulador siguiendo el patrón Modelo-Vista-Controlador.

La clase **Equipo** es la base de donde tiene varias referencias. De ahí heredan **Cellular**, **EloTelTag** y **Tablet**, cada uno con su propia lógica: los celulares reportan posiciones, y los tags y tablets pueden emitir señales de búsqueda. Cada modelo tiene su vista: **CellularView**, **EloTelTagView** y **TabletView**. Las vistas se encargan de dibujar los equipos en el mapa, mostrar la animación del radar y ofrecer el menú FindMy. **CellularView** y **TabletView** necesitan acceso a **ETNube** para poder mostrar las ubicaciones en la ventana FindMy.

Territorio hace de controlador: tiene la lista de equipos, los mueve, controla los tiempos de bip y detecta cuándo un tag o tablet está cerca de un celular para generar los reportes.

ETNube es donde se guardan todas las posiciones reportadas, usando un **HashMap** que asocia cada dueño con sus dispositivos.

TerritorioView muestra la imagen de fondo y contiene todas las vistas de los equipos.

Y **Stage4** es la clase principal, este lee el archivo de configuración, crea los objetos, configura el **Timeline** y levanta la interfaz que se observa al ejecutar los códigos.

1.3. Dificultades y Soluciones

1. **La ventana FindMy no se actualizaba.** En la Etapa 3 la ventana FindMy mostraba las posiciones solo al momento de abrirla, pero después quedaba congelada. Para arreglarlo, le pusimos un `Timeline` que cada 1 segundo limpia el contenido y lo vuelve a armar consultando `ETNube`. También configuramos `setOnCloseRequest` para que el `Timeline` se detenga al cerrar la ventana.
2. **El programa crasheaba si no encontraba el archivo de sonido.** Al cargar `bip.wav` con `AudioClip`, si el archivo no estaba en la carpeta el programa tiraba una excepción y se caía. Lo solucionamos envolviendo la carga en un `try-catch`: si no encuentra el archivo, `bipSound` queda en `null` y antes de reproducir siempre verificamos con `if (bipSound != null)`. Esto debido a que si un ítem está cerca, este genera un radio de color azul si es que está cercano a un dispositivo gracias al tag no necesariamente se escucha si no se tiene el archivo del sonido.
3. **Los contadores de bip de tags y tablets se mezclaban.** Al principio usamos un solo contador para tags y tablets, lo que hacía que ambos emitieran señal al mismo tiempo. La solución fue crear contadores separados dentro de `Territory`, cada uno con su propio periodo de 4 y 5 segundos respectivamente, para que los periodos de tiempos fueran individuales.